

Template Literals & Truthy/Falsy Values

Recap of the Day 4 video: building strings with backticks and `${ }` instead of messy concatenation, and the six falsy values that decide what an if-statement really does.

Template literals

Template literals use **backticks** (```) instead of quotes. Inside them, `${ }` drops any expression straight into the string — and line breaks are kept as-is.

```
const name = "Palla";
// old way – concatenation:
const msg1 = "Hi " + name + ", you scored " + (40 + 5) + "!";
// new way – template literal:
const msg2 = `Hi ${name}, you scored ${40 + 5}!`;
const multi = `line one
line two`; // real line break, no \n needed
```

- In automation you will use these constantly: ``Test case ${id} failed on ${browser}``.

Truthy & falsy values

When JavaScript needs a yes/no answer — like inside `if (value)` — every value converts to true or false. Only **six values are falsy**; everything else is truthy.

```
// the SIX falsy values – memorise them:
false  0  ""  null  undefined  NaN
if ("hello") { } // runs – non-empty string is truthy
if (0)          { } // skipped – 0 is falsy
if ([])         { } // runs – empty array is TRUTHY!
if ({})         { } // runs – empty object is TRUTHY!
Boolean("")     // false – check any value yourself
!!"text"        // true  – the double-bang shortcut
```

Quick reference

Value	Truthy or falsy?
false, 0, "", null, undefined, NaN	The six falsy values — everything else is truthy
"0" (string zero)	Truthy — it is a non-empty string
[] (empty array)	Truthy — a common surprise
{ } (empty object)	Truthy — a common surprise
-1 (negative number)	Truthy — only 0 (and NaN) are falsy numbers

Practice exercises

Do these in VS Code, run them with Node, and bring questions to the next live Q&A.

1. Rewrite with a template literal: "User " + user + " has " + count + " messages".
2. Build a three-line address string using ONE template literal with real line breaks (no `\n`).
3. Predict truthy or falsy, then verify each with `Boolean(...)` in Node: `"0"`, `0`, `""`, `" "`, `[]`, `undefined`, `NaN`, `-1`.

Hint: Exactly four of these are falsy.

4. Write a function `check(value)` that prints ``${value} is truthy`` or ``${value} is falsy`` using a template literal.
5. Loop `i` from 1 to 5 and print ``Test case ${i} passed`` each time.

Bonus challenge — Write `greet(name)` that prints ``Hello, ${name}!`` — but when `name` is falsy (empty string, `undefined`...) it greets "Guest" instead. Hint: `name || "Guest"`.